📅 Day 1 – Ansible Basics + Setup + Core Understanding

🔹 1. What is Ansible? (Simple Understanding)

Ansible is a tool that helps you **automate tasks on multiple servers at once**.

👉 Imagine this:
You have 50 servers and you want to install Java on all of them.

❌ Without Ansible:

- Login to each server
- Install manually (very slow)

✅ With Ansible:

- Write one command
- It runs on all servers automatically

🔹 2. Why Companies Use Ansible

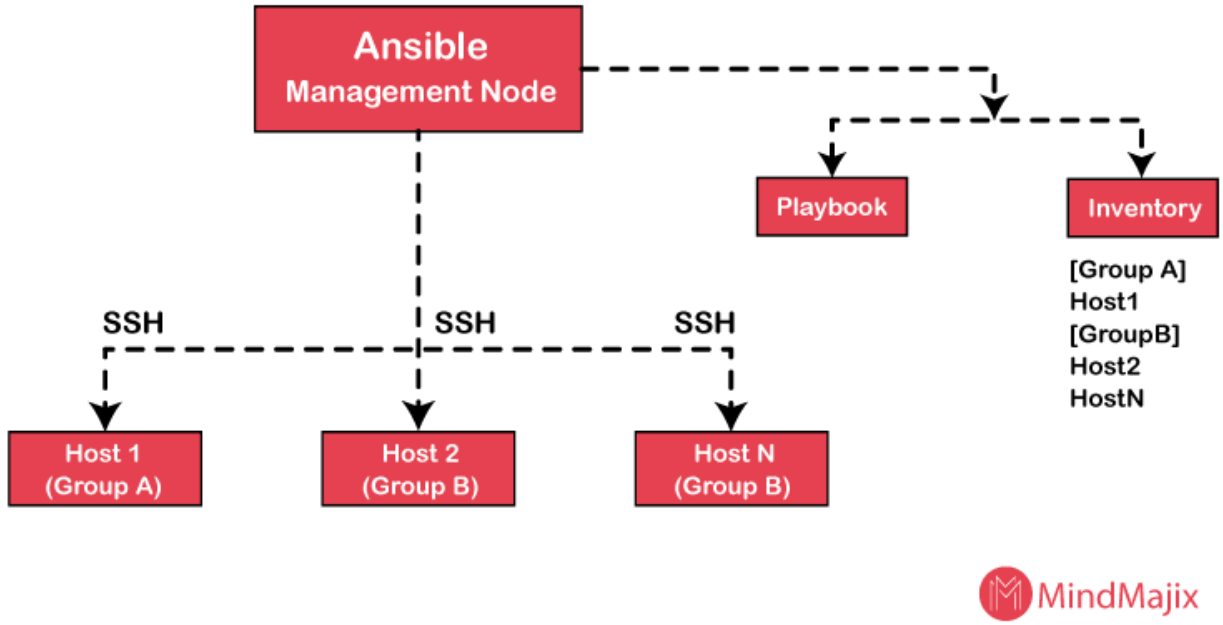
Companies use it for:

- Server setup (install Java, Nginx, Docker)
- Application deployment
- Configuration management
- CI/CD pipelines (with Jenkins)

👉 Real-world example:
When your Spring app is deployed, Ansible:

1. Installs Java
2. Copies your JAR
3. Starts the application

🔹 3. How Ansible Works (Core Idea)



Ansible follows a very simple and lightweight architecture where one main machine controls multiple target machines without needing to install anything extra on them. The machine where Ansible is installed is called the **Control Node**, and this is where you write your commands or playbooks. The systems on which tasks are executed are called **Managed Nodes**, and these can be servers, virtual machines, or even cloud instances. The communication between the control node and managed nodes happens using **SSH**, which means Ansible does not require any agent or software to be installed on the target systems, making it very easy to set up and use.

Now, if we understand the flow, it becomes very straightforward. First, you write a command or a playbook on the control node, which defines what tasks you want to perform, such as installing software or restarting a service. Then Ansible connects to the managed nodes using SSH, securely and automatically. After establishing the connection, it executes the tasks one by one on those machines. Once the tasks are completed, Ansible collects the output and shows it back on your control node so you can see whether everything was successful or if something failed.

For example, imagine you are a Java developer working in a company and you need to install Java on 10 servers. Instead of logging into each server manually and running installation commands, you can write a simple Ansible playbook on your control node. When you run it, Ansible will connect to all 10 servers via SSH, install Java on each one, and then return the

result. This saves time, reduces human error, and ensures all servers are configured in the same way.

Ansible follows a **very simple architecture**:

🧠 Components:

1. **Control Node**
   - Your machine (where Ansible is installed)
2. **Managed Nodes**
   - Target servers (where tasks run)
3. **SSH**
   - Used to connect (no agent needed)

🔁 Flow:

1. You write a command/playbook
2. Ansible connects via SSH
3. Executes tasks
4. Returns output

👉 No agent installation needed → **BIG advantage**

🔹 4. Important Concept: Agentless

👉 **What does agentless mean?**

Unlike tools like Puppet:

- ❌ No software installed on servers
- ✅ Uses SSH directly

👉 That's why Ansible is simple and fast to start.

🔹 5. Installation (Mac / Linux / WSL)

Since you're using Mac / Linux-like environment, do:

✅ Install via pip (recommended)

```
pip install ansible
```

✅ Verify installation

```
ansible --version
```

🔹 6. First Hands-On (VERY IMPORTANT)

Let's simulate servers using **localhost**.

Step 1: Create Inventory File

Create file: `inventory.ini`

```
localhost ansible_connection=local
```

Step 2: Test Connection

```
ansible all -i inventory.ini -m ping
```

👉 Output:

```
localhost | SUCCESS => {
    "ping": "pong"
}
```

✔️ This means Ansible is working

🔹 7. First Command (Ad-hoc Command)

Run:

```
ansible all -i inventory.ini -m command -a "echo Hello Siddharth"
```

👉 Output:

Hello Siddharth

This command is an **Ansible ad-hoc command** used to run a simple shell instruction on multiple target machines at once, without writing a playbook. Let's understand it step by step in a clear way.

The command starts with `ansible all`, which means Ansible will target **all the managed nodes** defined in your inventory. The `-i inventory.ini` part tells Ansible where to find the list of servers (your inventory file). This file contains the IP addresses or hostnames of the machines you want to control.

Next, `-m command` specifies that Ansible should use the **command module**. This module is used to run basic commands on the remote machines, but it does not process shell features like pipes (`|`), redirection (`>`), or variables. It simply executes the command as it is.

Then, `-a "echo Hello Siddharth"` is the argument passed to the command module. This means Ansible will run the `echo Hello Siddharth` command on each managed node. The `echo` command just prints the text, so each server will return the message **"Hello Siddharth"**.

Putting it all together, this command tells Ansible:
"Go to all the servers listed in `inventory.ini`, connect via SSH, run the command `echo Hello Siddharth`, and show me the output."

For example, if your `inventory.ini` contains 3 servers, Ansible will connect to all 3 machines and you will see output like:

```
server1 | SUCCESS | rc=0 >>
Hello Siddharth

server2 | SUCCESS | rc=0 >>
Hello Siddharth
```

```
server3 | SUCCESS | rc=0 >>
Hello Siddharth
```

In a real-world scenario, you might use a similar command to quickly check system status, verify connectivity, or print some debug information across multiple servers at once, instead of logging into each machine manually.

An **Ad-hoc Command in Ansible** is a simple, one-line command that you run directly from the terminal to perform a quick task on one or more managed nodes without writing a full playbook. It is mainly used for **immediate actions**, testing, or quick operations where creating a playbook would be unnecessary and time-consuming. These commands are not stored or reusable like playbooks; they are executed instantly and are best suited for small, one-time tasks.

In terms of how it works, when you run an ad-hoc command, Ansible uses the control node to connect to the managed nodes via SSH, executes the specified module (like ping, copy, or command), and then returns the output back to your terminal. It follows the same architecture as playbooks but in a much faster and more direct way, without requiring YAML files.

For example, suppose you want to check if all your servers are reachable. Instead of writing a playbook, you can simply run:

ansible all -m ping

Here, **all** means all managed nodes, and **-m ping** tells Ansible to use the ping module. Ansible will connect to each server via SSH and return a success or failure response.

Another practical example is if you want to install a package like Git on all servers quickly:

ansible all -m apt -a "name=git state=present"

This command directly installs Git on all managed nodes without writing a playbook.

So in short, an **ad-hoc command is like a shortcut** in Ansible for quick, one-time operations, while playbooks are used for more complex, repeatable automation tasks.

🔹 8. Key Concepts You Learned Today

Concept	Meaning
Control Node	Your system
Managed Node	Target servers
Inventory	List of servers
Module	Work unit (like ping, command)
SSH	Connection method

🔹 9. Important Interview Questions (Day 1)

👉 Q1: What is Ansible?

An automation tool used for configuration management, deployment, and orchestration.

👉 Q2: How Ansible works?

It connects to remote machines via SSH and executes tasks using modules.

👉 Q3: Why Ansible is agentless?

Because it uses SSH instead of installing agents on target machines.

👉 Q4: What is a module?

A module is a small program that performs a specific task (e.g., ping, install package).

🔹 10. Real-World Insight (VERY IMPORTANT)

In companies:

- Dev pushes code → Git
- Jenkins triggers pipeline
- Ansible:
  - sets up servers
  - deploys app
  - restarts services

🎯 Your Task (Do This Now)

1. Install Ansible
2. Create `inventory.ini`
3. Run:
    - `ansible all -m ping`
    - `ansible all -m command -a "date"`

📅 Day 2 – Inventory Deep Dive + Ad-hoc Commands (Real Usage)

Today is **very important**, because this is how you actually control multiple servers in real companies.

🔹 1. What is Inventory (Deep Understanding)

👉 Inventory = **list of servers where Ansible runs tasks**

Think of it like:

> "Where should my command go?"

🧠 Types of Inventory

1. Static Inventory (MOST COMMON for beginners)

```
[web]
192.168.1.10
192.168.1.11

[db]
192.168.1.20
```

👉 Here:

- `web` = group of web servers
- `db` = database servers

2. With Variables

[web]

```
192.168.1.10 ansible_user=ubuntu ansible_port=22
```

👉 This tells:

- which user to login
- which port to use

This line is an **entry inside an Ansible inventory file**, and it is used to define how Ansible should connect to a specific managed node (server). It contains the **IP address of the server** along with some connection details like username and port.

The first part, `192.168.1.10`, represents the **target machine's IP address**, which means this is the server where Ansible will execute tasks. When you run any command or playbook, Ansible will use this IP to identify and connect to that machine.

The next part, `ansible_user=ubuntu`, tells Ansible which **username to use while logging into the server via SSH**. In this case, Ansible will try to connect using the `ubuntu` user. This is very common in cloud environments like AWS EC2, where the default user is often `ubuntu`.

Then, `ansible_port=22` specifies the **SSH port number**. By default, SSH runs on port 22, so even if you don't mention it, Ansible assumes port 22. However, if your server is configured to use a different port (for security reasons), you would explicitly define it here.

Putting it all together, this line tells Ansible:
"Connect to the server with IP `192.168.1.10`, use the username `ubuntu`, and connect through SSH on port `22`."

For example, when you run an ad-hoc command like:

```
ansible all -i inventory.ini -m ping
```

Ansible will read this line, connect to `192.168.1.10` using `ubuntu@192.168.1.10:22`, and then execute the ping module.

In a real-world scenario, if you have multiple servers, your inventory file will contain multiple such lines, and Ansible will use these details to automate tasks across all those machines without you manually logging into each one.

3. Local Testing (your case)

```
[local]
localhost ansible_connection=local
```

🔹 2. Group Concept (VERY IMPORTANT)

Groups help you target multiple servers.

👉 Example:

```
ansible web -m ping
```

✔️ Runs only on `web` servers
✔️ Not on database servers

🔹 3. Ad-hoc Commands (Real Power)

👉 Ad-hoc = **one-time command execution**

🔥 Most Used Commands

✅ 1. Ping servers

```
ansible all -i inventory.ini -m ping
```

✅ 2. Check uptime

```
ansible all -i inventory.ini -m command -a "uptime"
```

✅ 3. Install package (Linux)

```
ansible all -i inventory.ini -m apt -a "name=nginx state=present" --become
```

👉 `--become` = sudo access

This command is an **Ansible ad-hoc command used to install a package (Nginx) on all target servers with administrative privileges**. It is a very common real-world command used for quickly setting up software across multiple machines.

The command starts with `ansible all`, which means Ansible will target **all the managed nodes** listed in your inventory file. The `-i inventory.ini` tells Ansible where to find those server details. So, all the machines defined inside `inventory.ini` will be selected for this operation.

Next, `-m apt` specifies that Ansible will use the **apt module**, which is specifically designed for Debian-based systems like Ubuntu. This module helps in managing packages such as installing, updating, or removing software.

Then, `-a "name=nginx state=present"` provides the arguments to the apt module. Here, `name=nginx` means the package you want to manage is **Nginx**, and `state=present` means Ansible should ensure that Nginx is installed. If Nginx is not already installed, it will install it; if it is already installed, Ansible will do nothing, which makes the operation **idempotent** (safe to run multiple times).

Finally, `--become` is a very important part of this command. It tells Ansible to **execute the task with elevated privileges (like sudo)** because installing software requires administrative access. Without this, the command would likely fail due to permission issues.

Putting it all together, this command means:
“Connect to all servers listed in `inventory.ini`, log in via SSH, switch to sudo/root privileges, install Nginx if it is not already installed, and return the result.”

For example, imagine you are deploying a web application and need Nginx installed on 5 servers. Instead of logging into each server and running `sudo apt install nginx`, you just run this one command. Ansible will automatically connect to all servers, install Nginx where needed, and show you the output, saving time and ensuring consistency across all machines.

✅ 4. Create file

```
ansible all -i inventory.ini -m file -a "path=/tmp/test.txt state=touch"
```

This command is an **Ansible ad-hoc command used to create a file on all target servers** in a simple and automated way.

The command starts with `ansible all`, which means Ansible will run this task on **all the managed nodes** listed in your `inventory.ini` file. The `-i inventory.ini` tells Ansible where to find the list of those servers.

Next, `-m file` specifies that Ansible should use the **file module**. This module is used to manage files and directories on remote machines, such as creating, deleting, or changing permissions.

Then, `-a "path=/tmp/test.txt state=touch"` provides the arguments to the file module. Here, `path=/tmp/test.txt` defines the location and name of the file you want to manage. The `state=touch` means Ansible should **create the file if it does not exist**, and if it already exists, it will simply update its timestamp (just like the Linux `touch` command).

Putting it all together, this command tells Ansible:
"Connect to all servers in `inventory.ini`, go to each one, and ensure that a file named `/tmp/test.txt` exists."

For example, if you have 3 servers, Ansible will connect to each server via SSH and create the file `/tmp/test.txt` on all of them. The output will show which servers had changes (file created) and which already had the file.

In a real-world scenario, you might use this to quickly create log files, marker files, or temporary files across multiple servers without logging into each machine manually.

✅ 5. Copy file

```
ansible all -i inventory.ini -m copy -a "src=test.txt dest=/tmp/test.txt"
```

This command is an **Ansible ad-hoc command used to copy a file from your control node to all managed nodes**.

The command begins with `ansible all`, which means Ansible will execute this task on **all the servers** listed in your `inventory.ini` file. The `-i inventory.ini` specifies where Ansible should read the list of target machines from.

Next, `-m copy` tells Ansible to use the **copy module**, which is specifically designed to transfer files from the control node (your machine) to the managed nodes (remote servers). This module ensures that the file is copied safely and consistently across all systems.

Then, `-a "src=test.txt dest=/tmp/test.txt"` provides the arguments. Here, `src=test.txt` means the source file is located on your **control node** (the machine where you are running Ansible). The `dest=/tmp/test.txt` specifies the destination path on the **remote servers**, meaning the file will be copied to `/tmp/test.txt` on each managed node.

Putting it all together, this command tells Ansible:
"Take the file `test.txt` from my local machine, connect to all servers listed in `inventory.ini` via SSH, and copy this file to `/tmp/test.txt` on each server."

For example, if you have 5 servers and you want to distribute a configuration file or a script, instead of manually copying it using `scp` multiple times, you can run this single command. Ansible will handle the file transfer to all servers and show you which ones were updated successfully.

In a real-world scenario, this is commonly used to deploy configuration files, scripts, or static resources across multiple machines quickly and consistently.

🔹 4. Important Modules (Must Know)

Module	Use
ping	Test connectivity
command	Run commands
shell	Run shell commands
copy	Copy files
file	Create/delete files
apt/yum	Install packages

🔹 5. Command vs Shell (Interview Favorite)

The difference between the **command module** and the **shell module** in Ansible is a very common interview question, and the key idea is how they execute commands on remote machines.

The **command module** is used to run simple commands on the target servers in a safe and predictable way. It does not use a shell, which means it does not understand shell-specific features like pipes (`|`), redirection (`>`), environment variables (`$HOME`), or operators like `&&`. Because of this, it is more secure and is the **recommended default choice** whenever you just need to run straightforward commands. For example, if you run `ansible all -m command -a "ls /tmp"`, Ansible will simply list files in the `/tmp` directory on all servers without interpreting anything extra.

On the other hand, the **shell module** runs commands through a shell (like `/bin/bash`), which means it supports all shell features such as piping, redirection, variable expansion, and chaining commands. This makes it more powerful but slightly less secure, because shell interpretation can introduce risks if inputs are not controlled properly. For example, if you run `ansible all -m shell -a "cat /etc/passwd | grep root"`, this will work because the shell understands the pipe (`|`) and executes it correctly.

To understand this with a real-world example, imagine you want to check if a service is running and then log the result into a file. With the command module, you cannot combine commands like `ps -ef | grep nginx > output.txt` because it does not support pipes or redirection. But with the shell module, you can do this easily since it behaves like a normal Linux terminal.

So in short, the **command module is safer and should be your default choice**, while the **shell module is used only when you need shell-specific features** like pipes, redirection, or complex command chaining.

👉 **command module**

- Safe
- No shell features

👉 **shell module**

- Supports:
  - pipes (`|`)
  - redirection (`>`)
  - variables

Example:

```
ansible all -m shell -a "ls | grep test"
```

This command is an **Ansible ad-hoc command using the shell module to run a piped Linux command on all target servers**.

The command starts with `ansible all`, which means Ansible will execute this on **all managed nodes** defined in your inventory (default or specified). Then `-m shell` tells Ansible to use the **shell module**, which runs the command inside a shell (like `/bin/bash`). This is important because shell allows advanced features like pipes (`|`), redirection (`>`), and variables.

Next, `-a "ls | grep test"` is the actual command being executed on each server. The `ls` command lists all files in the current directory, and `grep test` filters that list to show only files that contain the word **"test"** in their name. The pipe (`|`) connects these two commands, which is something only the **shell module can handle**, not the command module.

Putting it all together, this command tells Ansible:
“Connect to all servers via SSH, run the shell command `ls | grep test`, and return the filtered list of files that contain the word ‘test’.”

For example, if a server has these files:

test.txt

data.txt

mytest.log

hello.sh

The output will be:

test.txt

mytest.log

In a real-world scenario, you might use this to quickly search for specific log files, configuration files, or any files matching a pattern across multiple servers. This is especially useful when debugging or verifying deployments.

🔹 6. Real-World Scenario (Very Important)

👉 Imagine you are in a company:

You need to:

- Check disk space on 100 servers

Instead of logging manually:

```
ansible all -m command -a "df -h"
```

✔️ Done in seconds

🔹 7. Common Errors + Debugging

❌ Error: Permission denied

👉 Fix:

```
--ask-pass
```

or use SSH keys

❌ Error: Host unreachable

👉 Check:

- IP correct?
- SSH working?

❌ Error: sudo required

👉 Use:

--become

🔹 8. Important Interview Questions

👉 Q1: What is inventory?

Inventory is a file containing list of servers and groups.

👉 Q2: What are ad-hoc commands?

One-time commands executed without playbooks.

👉 Q3: Difference between command and shell?

command	shell
No shell features	Supports pipes
Safer	More flexible

👉 Q4: What is grouping in inventory?

Grouping allows running tasks on multiple servers together.

🔹 9. Real DevOps Flow

In real companies:

- Inventory is huge (100–1000 servers)
- Stored dynamically (AWS, cloud)

Example:
Using Amazon Web Services inventory:

- Automatically fetch EC2 instances
- No manual IP entry

🎯 Your Task (DO THIS)

Create `inventory.ini` like:

[local]

localhost ansible_connection=local

Then run:

```
ansible local -i inventory.ini -m command -a "date"
ansible local -i inventory.ini -m file -a "path=/tmp/day2.txt state=touch"
ansible local -i inventory.ini -m shell -a "echo Hello > /tmp/day2.txt"
```

📅 Day 3 – Playbooks (Core of Ansible)

🔹 1. What is a Playbook?

An **Ansible Playbook** is a YAML file where you define a set of tasks that you want to execute on managed nodes in an organized, reusable, and automated way. Unlike ad-hoc commands (which are one-time executions), playbooks are used for **repeatable automation**, such as server setup, application deployment, or configuration management. It allows you to describe *what you want to achieve* rather than writing step-by-step commands manually every time.

Now let's understand this with a simple example playbook and explain **each line clearly**.

```yaml
---
- name: Install and start Nginx
  hosts: all
  become: yes

  tasks:
    - name: Install nginx package
      apt:
        name: nginx
        state: present

    - name: Start nginx service
      service:
        name: nginx
        state: started
```

The first line `---` indicates the **start of a YAML file**. It is optional but commonly used for clarity and standard practice.

The next line `- name: Install and start Nginx` defines the **name of the play**. A play is basically a group of tasks. This name is just for humans to understand what this playbook is doing when it runs.

The line `hosts: all` specifies **which machines this playbook will run on**. In this case, it means all the servers defined in your inventory file.

The line `become: yes` tells Ansible to **use elevated privileges (sudo/root)** while executing the tasks. This is required because installing software and managing services usually need admin access.

Next comes `tasks:`, which is a section that contains a list of actions you want Ansible to perform. Everything under this is a task.

The first task starts with `- name: Install nginx package`, which is just a description of what this task does.

Then `apt:` specifies the **module** being used. Since this is an Ubuntu/Debian system, the apt module is used to manage packages.

Inside it, `name: nginx` tells Ansible which package to install, and `state: present` ensures that **nginx is installed**. If it is already installed, Ansible will not reinstall it, which makes it idempotent.

The second task starts with `- name: Start nginx service`, again just a descriptive name.

Then `service:` is the module used to manage system services.

Inside it, `name: nginx` specifies the service name, and `state: started` ensures that the **nginx service is running**. If it is already running, Ansible will leave it as is.

Putting it all together, this playbook tells Ansible:
"Go to all servers, use sudo access, install Nginx if it's not already installed, and make sure the Nginx service is running."

For a real-world example, imagine you are deploying a web application across 10 servers. Instead of manually installing Nginx and starting the service on each server, you just run this playbook once, and Ansible will handle everything automatically, consistently, and safely across all machines.

👉 A **Playbook** is a YAML file where you define:

- what to do
- where to do
- how to do

Think of it like:

> "Step-by-step automation script for servers"

🧠 Real Example

Instead of running 5 commands manually:

- install nginx
- start service

- create file

👉 You write **one playbook**, and Ansible does everything.

Real World Production PlayBook:

A **real-world production Ansible playbook** is usually more structured and handles multiple things together like package installation, configuration, service management, and ensuring the system is in the desired state. Let's take a practical example of deploying a **Java-based web application with Nginx as a reverse proxy**, which is very common in product-based companies.

Below is a production-style playbook, and I will explain **each part clearly in paragraph format** so you can visualize how it works in real environments.

```yaml
---
- name: Deploy Java Application with Nginx
  hosts: webservers
  become: yes
  vars:
    app_name: myapp
    app_port: 8080

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes

    - name: Install required packages
      apt:
        name:
          - openjdk-17-jdk
          - nginx
        state: present

    - name: Copy application JAR file
      copy:
        src: /home/ubuntu/myapp.jar
        dest: /opt/myapp.jar
        mode: '0755'

    - name: Create systemd service for app
      copy:
        dest: /etc/systemd/system/myapp.service
```

```
      content: |
        [Unit]
        Description=My Java App
        After=network.target

        [Service]
        ExecStart=/usr/bin/java -jar /opt/myapp.jar
        Restart=always
        User=ubuntu

        [Install]
        WantedBy=multi-user.target

  - name: Reload systemd
    command: systemctl daemon-reexec

  - name: Enable and start app service
    service:
      name: myapp
      enabled: yes
      state: started

  - name: Configure Nginx reverse proxy
    copy:
      dest: /etc/nginx/sites-available/myapp
      content: |
        server {
            listen 80;
            location / {
                proxy_pass http://localhost:8080;
            }
        }

  - name: Enable Nginx config
    file:
      src: /etc/nginx/sites-available/myapp
      dest: /etc/nginx/sites-enabled/myapp
      state: link

  - name: Restart Nginx
    service:
      name: nginx
      state: restarted
```

Now let's understand this like a real production flow.

The playbook starts with `---`, which simply marks the beginning of the YAML file. Then `- name: Deploy Java Application with Nginx` defines the overall purpose of this play, which helps during execution logs to understand what is happening.

The line `hosts: webservers` means this playbook will run only on servers grouped as **webservers** in your inventory. In production, servers are usually grouped like `webservers`, `dbservers`, etc., instead of using `all`.

The `become: yes` ensures that all tasks run with **sudo privileges**, which is required for installing packages, creating system services, and modifying system configurations.

The `vars` section defines variables like `app_name` and `app_port`. This makes the playbook reusable and easier to maintain, because you can change values in one place instead of editing multiple lines.

Now the tasks begin. The first task updates the package cache using `apt`, ensuring the system has the latest package information before installing anything.

The next task installs required software like **Java (OpenJDK 17)** and **Nginx**, which are necessary to run the application and expose it to users.

Then, the playbook copies the application JAR file from the control node to the server at `/opt/myapp.jar`. This is how your application code is deployed to servers.

Next, a **systemd service file** is created. This is very important in production because instead of manually running `java -jar`, the application is managed as a service. This ensures it starts automatically on reboot and can be controlled using system commands.

After creating the service file, Ansible reloads systemd using `systemctl daemon-reexec` so the system recognizes the new service.

Then, the playbook enables and starts the application service. This ensures the app is always running and also starts automatically after system restarts.

Next, Nginx is configured as a **reverse proxy**. This means users will access the app via port 80 (HTTP), and Nginx will internally forward requests to the Java application running on port 8080.

The following task creates a symbolic link to enable the Nginx configuration, which is a standard practice in Linux systems.

Finally, Nginx is restarted so that the new configuration takes effect.

In a real-world scenario, this playbook represents a **complete deployment pipeline step**. For example, when a developer pushes code, your CI/CD tool (like Jenkins) can trigger this playbook to automatically deploy the application on multiple servers, configure everything, and make the application live without any manual intervention.

🔹 2. YAML Basics (VERY IMPORTANT)

Playbooks are written in YAML.

🔑 Rules:

- Use indentation (spaces, not tabs)
- Key-value format
- Lists use `-`

Example YAML:

```
- name: My First Play
  hosts: localhost
  tasks:
    - name: Print message
      debug:
        msg: "Hello Ansible"
```

🔹 3. Playbook Structure (Must Understand)

Every playbook has this structure:

```
- name: Play name
  hosts: target_servers
  become: yes/no
  tasks:
    - name: Task 1
      module:
        parameter: value
```

🔑 Key Parts:

Field	Meaning
name	Description
hosts	Where to run
become	sudo access
tasks	List of actions

🔹 4. Your First Real Playbook (Hands-on)

Step 1: Create file `playbook.yml`

```yaml
- name: My first playbook
  hosts: local
  become: no

  tasks:
    - name: Create a file
      file:
        path: /tmp/ansible_day3.txt
        state: touch

    - name: Write content
      copy:
        content: "Hello from Ansible Playbook"
        dest: /tmp/ansible_day3.txt
```

Step 2: Run Playbook

```
ansible-playbook -i inventory.ini playbook.yml
```

✅ Output Meaning

You'll see:

- `ok` → already correct
- `changed` → something updated

🔹 5. Important Concept: Idempotency (VERY IMPORTANT)

👉 This is a **favorite interview question**

🧠 Meaning:

Running the same playbook multiple times gives the same result.

Example:

- First run → creates file
- Second run → nothing changes

✔️ No duplication
✔️ No errors

🔹 6. Modules in Playbooks

In Ansible playbooks, **modules are the building blocks that actually perform the work on managed nodes**. A playbook itself is just a structured set of instructions written in YAML, but the real execution—like installing software, copying files, starting services, or creating users—is done by modules. You can think of modules as **predefined tools or functions** that Ansible provides so you don't have to write complex scripts manually.

When a playbook runs, each task calls a specific module, and that module ensures the system reaches the desired state. One important thing about modules is that they are **idempotent**, which means if the system is already in the desired state, the module will not make unnecessary changes. This is very important in production because it avoids repeated or harmful operations.

Let's understand this with a small example inside a playbook:

```yaml
- name: Example Playbook
  hosts: all
  become: yes

  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present

    - name: Copy config file
      copy:
        src: nginx.conf
        dest: /etc/nginx/nginx.conf

    - name: Start nginx
      service:
        name: nginx
        state: started
```

Now let's understand how modules are used here in a real sense.

In the first task, `apt` is the module. It is responsible for managing packages on Debian-based systems. Instead of writing a command like `sudo apt install nginx`, you simply declare `state: present`, and the module handles everything for you.

In the second task, `copy` is the module. It copies a file from your control node to the managed node. This is commonly used in real projects to deploy configuration files or application code.

In the third task, `service` is the module. It ensures that a service (like nginx) is started, stopped, or restarted. In production, this is critical because services must always be in the correct state.

In real-world scenarios, modules are used everywhere. For example, if you are deploying a Java application, you might use the `apt` module to install Java, the `copy` module to transfer your JAR file, the `file` module to create directories, and the `service` module to run your application as a service.

So in simple terms, **modules are like ready-made actions**, and a playbook is just a sequence of those actions organized in a proper flow. Without modules, Ansible would not be able to perform any real task on the servers.

Same modules from Day 2 are used here:

🔥 Most Used:

✅ File module

```
- name: Create file
  file:
    path: /tmp/test.txt
    state: touch
```

✅ Copy module

```
- name: Copy content
  copy:
    content: "Hello"
    dest: /tmp/test.txt
```

✅ Package install (Linux)

```
- name: Install nginx
  apt:
    name: nginx
    state: present
```

🔹 7. Become (Sudo Access)

👉 Needed when installing packages or system-level changes

become: yes

🔹 8. Real-World Example (VERY IMPORTANT)

🚀 Deploy Web Server

```
- name: Setup Web Server
  hosts: local
  become: yes

  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present

    - name: Start nginx
      service:
        name: nginx
        state: started
```

👉 This is what companies do automatically using
Ansible + Jenkins

🔹 9. Important Interview Questions

👉 Q1: What is a playbook?

A YAML file that defines automation tasks in sequence.

👉 Q2: What is idempotency?

Running the same playbook multiple times gives the same result without duplication.

👉 Q3: Difference between ad-hoc and playbook?

Ad-hoc	Playbook
One-time	Reusable
Quick	Structured
Not scalable	Production ready

👉 Q4: What is become?

Used to execute tasks with sudo/root privileges.

🔹 10. Real DevOps Flow (Visualization)

In companies:

1. Code pushed to Git
2. Jenkins triggers pipeline
3. Ansible playbook runs:
    - install dependencies
    - deploy app
    - restart services

🎯 Your Task (DO THIS)

1. Create this playbook:

- Create file
- Write content

2. Run twice:

ansible-playbook -i inventory.ini playbook.yml

👉 Observe:

- First time → changed
- Second time → ok

📅 Day 4 – Variables + Facts + Conditionals

🔹 1. Why Do We Need Variables?

👉 Without variables:

- Everything is hardcoded
- Not reusable

👉 With variables:

- Dynamic automation
- Reusable playbooks
- Environment-based execution (dev / prod)

🧠 Example

Instead of:

```
name: nginx
```

Use:

```
name: "{{ package_name }}"
```

✔️ Now you can change value anytime

🔹 2. Types of Variables in Ansible

✅ 1. Playbook Variables

```
- name: Variable Example
  hosts: local

  vars:
    package_name: nginx

  tasks:
    - name: Install package
      apt:
        name: "{{ package_name }}"
        state: present
```

✅ 2. Inventory Variables

[local]

```
localhost ansible_connection=local package_name=nginx
```

✅ 3. Extra Variables (Highest Priority)

```
ansible-playbook playbook.yml -e "package_name=nginx"
```

In Ansible, **variables are used to store values that can be reused across your playbooks**, making your automation more flexible, dynamic, and maintainable. Instead of hardcoding values like usernames, ports, or file paths, you define variables and use them wherever needed. In real-world production environments, variables are heavily used to manage different environments like **dev, staging, and production** without changing the core playbook.

Now, let's understand the **types of variables in Ansible** in a clear and practical way.

First, **Playbook Variables (vars)** are defined directly inside a playbook. These are simple and commonly used when the values are specific to that playbook. For example:

```
- name: Example Playbook
  hosts: all
  vars:
    app_port: 8080
    app_name: myapp

  tasks:
    - name: Print app name
      debug:
        msg: "Application name is {{ app_name }}"
```

Here, `app_name` and `app_port` are defined inside the playbook and can be used anywhere within it. This is useful when the values are fixed for that particular automation.

Next, **Inventory Variables** are defined inside the inventory file. These are used when different servers need different configurations. For example:

```
192.168.1.10 ansible_user=ubuntu app_env=dev
192.168.1.11 ansible_user=ubuntu app_env=prod
```

Here, each server has its own variable `app_env`. This is very useful in production when different servers behave differently based on environment.

Then, we have **Group Variables (group_vars)**, which are defined for a group of servers. For example, if you have a group called `webservers`, you can create a file like `group_vars/webservers.yml`:

```
app_port: 8080
```

This means all servers in the `webservers` group will use this variable. In real-world projects, this is commonly used to manage configurations for entire environments like all web servers or all database servers.

Similarly, **Host Variables (host_vars)** are defined for a specific server. For example, `host_vars/192.168.1.10.yml`:

```
app_port: 9090
```

This overrides group-level or playbook-level variables for that specific machine. This is useful when one server needs special configuration.

Another important type is **Extra Variables**, which are passed at runtime using the command line. For example:

```
ansible-playbook deploy.yml -e "app_port=9090"
```

These variables have **highest priority** and are often used in CI/CD pipelines where values change dynamically.

Then we have **Registered Variables**, which store the output of a task so it can be used later. For example:

```
- name: Check nginx status
  command: systemctl status nginx
  register: nginx_status

- name: Print status
  debug:
    msg: "{{ nginx_status.stdout }}"
```

Here, the output of the first task is saved and reused in the next task. This is very useful in real-world debugging and conditional logic.

Finally, there are **Environment Variables**, which come from the system environment. These can be accessed using Ansible facts or lookup functions and are useful when integrating with system-level configurations.

In a real-world scenario, imagine you are deploying the same application to dev and production environments. Instead of writing two different playbooks, you use variables like `app_port`, `db_host`, or `env_type` and change their values using inventory, group_vars, or extra variables. This makes your automation reusable, scalable, and production-ready.

So in simple terms, **variables in Ansible help you avoid hardcoding, improve reusability, and manage different environments efficiently**, which is why they are a critical part of any real-world Ansible project.

🔹 3. Variable Precedence (Interview Question)

👉 Priority (high → low):

1. Extra vars
2. Playbook vars
3. Inventory vars

In Ansible, variables follow a **precedence order**, meaning if the same variable is defined in multiple places, the one with **higher priority overrides the others**. Understanding this is very important in real-world projects because variables can come from many sources.

Here is the **variable precedence from lowest to highest (short and structured):**

1. Role Defaults (Lowest priority)
These are default values defined inside roles. They are easily overridden and mainly used as fallback values.

2. Inventory Variables (group_vars / host_vars)
Variables defined in inventory files, group variables, and host variables. These are commonly used for environment-specific configurations like dev, staging, and production.

3. Playbook Variables (vars)
Variables defined inside the playbook itself. These override inventory variables.

4. Registered Variables
Variables created during runtime using `register`. These store task outputs and override previous values.

5. Extra Variables (Highest priority)
Variables passed via command line using `-e`. These always override everything else.

For example, if `app_port=8080` is defined in inventory, `app_port=9090` in playbook, and you run:

```
ansible-playbook deploy.yml -e "app_port=7070"
```

then the final value used will be **7070**, because extra variables have the highest precedence.

In real-world scenarios, this allows you to keep default values in code while overriding them dynamically during deployments, especially in CI/CD pipelines.

🔹 4. Facts (VERY IMPORTANT)

👉 Facts = **system information collected automatically**

In Ansible, **Facts are automatically collected information about the managed nodes**, and they are extremely important because they allow your playbooks to make **intelligent, dynamic decisions based on the system's actual state**. Instead of hardcoding values like OS type, IP address, memory, or CPU, Ansible gathers these details at runtime and stores them as variables called **facts**.

When a playbook starts, Ansible runs a special module called **setup**, which collects all system information from the target machine. These facts are then available as variables that you can use anywhere in your playbook. For example, facts include details like operating system (`ansible_os_family`), IP address (`ansible_default_ipv4.address`), hostname, memory, disk size, and many more.

Here is a simple example to understand how facts are used:

```
- name: Use facts example
  hosts: all

  tasks:
    - name: Print OS type
      debug:
        msg: "OS is {{ ansible_os_family }}"

    - name: Print IP address
      debug:
        msg: "IP is {{ ansible_default_ipv4.address }}"
```

In this example, Ansible automatically collects facts from each server and then prints the OS type and IP address. You don't need to manually define these values—they come directly from the system.

Now let's understand this with a real-world scenario. Suppose you are managing both Ubuntu and RedHat servers. Instead of writing separate playbooks, you can use facts to apply logic dynamically. For example, you can install packages using `apt` for Ubuntu and `yum` for RedHat based on the `ansible_os_family` fact. This makes your automation **smart and environment-aware**.

Another important point is that fact gathering can be controlled using `gather_facts`. By default, it is enabled, but you can disable it if not needed to improve performance:

gather_facts: no

In production, facts are heavily used for **conditional execution, debugging, dynamic configurations, and decision-making**. For example, you might use facts to deploy different configurations based on available memory or to assign roles based on hostname patterns.

So in simple terms, **facts are like real-time system information that Ansible collects automatically**, and they help you write powerful, flexible, and intelligent playbooks instead of static ones.

🔥 Examples of facts:

- OS type
- IP address
- Memory
- CPU

👉 Use facts:

```
- name: Show OS
  debug:
    msg: "{{ ansible_os_family }}"
```

👉 Get all facts:

```
ansible all -m setup
```

🔹 5. Register (Store Output)

👉 Used to store output of a task

Example:

```
- name: Check disk
  command: df -h
  register: disk_output

- name: Print disk
  debug:
    var: disk_output.stdout
```

🔹 6. Conditionals (when) – VERY IMPORTANT

👉 Makes decisions like programming

Example:

```
- name: Install nginx only on Debian
  apt:
    name: nginx
    state: present
  when: ansible_os_family == "Debian"
```

Another Example:

```
- name: Run only if disk space low
  debug:
    msg: "Low disk space!"
  when: disk_output.stdout.find("90%") != -1
```

🔹 7. Combining Everything (Real Example)

🚀 Smart Playbook

```
- name: Smart Playbook
  hosts: local
  become: yes

  vars:
    package_name: nginx

  tasks:
    - name: Install package
      apt:
        name: "{{ package_name }}"
        state: present
      when: ansible_os_family == "Debian"

    - name: Check disk usage
      command: df -h
      register: disk_output

    - name: Show warning if disk high
```

```
    debug:
      msg: "Disk usage is high!"
    when: disk_output.stdout.find("90%") != -1
```

🔹 8. Real-World Scenario (VERY IMPORTANT)

In companies using Ansible:

👉 Same playbook runs on:

- Dev server
- QA server
- Prod server

But behavior changes using:

- variables
- facts
- conditions

🧠 Example:

- Ubuntu → install `apt` package
- RedHat → install `yum` package

🔹 9. Important Interview Questions

👉 Q1: What are variables in Ansible?

Used to store values and make playbooks dynamic.

👉 Q2: What are facts?

Automatically gathered system information.

👉 Q3: What is register?

Stores output of a task for later use.

👉 Q4: What is when condition?

Used to execute tasks based on conditions.

👉 Q5: What is variable precedence?

Order in which variables override each other.

🔹 10. Real DevOps Flow

With Jenkins + Ansible:

1. Pipeline triggers
2. Pass environment:

-e "env=prod"

3. Playbook behaves differently:

- Dev → debug mode
- Prod → optimized

🎯 Your Task (DO THIS)

Step 1: Create playbook

- Install package using variable
- Print OS using fact
- Add condition

Step 2: Run:

ansible-playbook -i inventory.ini playbook.yml

📅 Day 5 – Roles + Templates + Handlers (REAL-WORLD LEVEL)

🔹 1. Why Day 5 is Important

Till now:

- You wrote playbooks (good ✅)

But in real companies:

- Code must be **modular, reusable, clean**

👉 That's why we use:

- Roles
- Templates
- Handlers

🔥 2. Roles (MOST IMPORTANT CONCEPT)

🔹 What is a Role?

👉 A **Role** is a structured way to organize Ansible code.

Instead of one big playbook:

- Break into smaller reusable components

An **Ansible Role** is a way to **organize your playbooks into a structured, reusable, and modular format**, especially useful in real-world production environments. Instead of writing one large playbook with everything inside it, roles allow you to break your automation into smaller, manageable parts like **web server setup, database setup, or application deployment**.

In simple terms, a role is like a **predefined folder structure** where you keep tasks, variables, files, templates, and handlers in a clean and standardized way. This makes your code easy to understand, reuse, and maintain, especially when working in teams or large projects.

A typical role structure looks like this:

```
my_role/
├── tasks/
│   └── main.yml
├── handlers/
│   └── main.yml
├── vars/
│   └── main.yml
├── defaults/
│   └── main.yml
├── files/
├── templates/
└── meta/
```

Now let's understand this in a practical way. The `tasks/main.yml` file contains the main list of tasks that the role will execute. The `handlers/main.yml` is used for actions like restarting services when something changes (for example, restarting Nginx after updating its config). The `vars` and `defaults` folders store variables, where defaults have lower priority and can be overridden. The `files` folder is used to store static files, and `templates` contains dynamic files (like configuration files using variables).

Here is how you use a role inside a playbook:

```yaml
- name: Use role example
  hosts: webservers
  become: yes

  roles:
    - nginx
```

This means Ansible will automatically go into the `nginx` role and execute everything defined inside it, mainly from `tasks/main.yml`.

For example, in a real-world production scenario, instead of writing one big playbook to install Nginx, configure it, deploy your app, and manage services, you create separate roles like `nginx`, `java`, and `app_deploy`. Then your main playbook simply calls these roles. This makes your automation **clean, scalable, reusable, and production-ready**.

So in short, a **role is a structured way to organize and reuse Ansible code**, making it easier to manage complex deployments in real-world environments.

🧠 Real Analogy

Think like Java:

- Playbook = Main class
- Role = Service classes

🔹 Role Structure

```
roles/
  web/
    tasks/
      main.yml
    handlers/
      main.yml
    templates/
    files/
    vars/
    defaults/
```

🔹 Create Role (Command)

```
ansible-galaxy init web
```

🔹 Example Role Task

👉 `roles/web/tasks/main.yml`

```
- name: Install nginx
  apt:
    name: nginx
    state: present

- name: Start nginx
  service:
    name: nginx
    state: started
```

🔹 Use Role in Playbook

```
- name: Use role
  hosts: local
  become: yes

  roles:
    - web
```

✔️ Clean
✔️ Reusable
✔️ Production ready

🔥 3. Templates (Dynamic Files using Jinja2)

🔹 What is Template?

👉 Template = dynamic file with variables

In Ansible, **Templates are used to create dynamic files using Jinja2**, which means you can generate configuration files with values that change based on variables, facts, or conditions. Instead of copying a static file to all servers, templates allow you to **customize the content for each server automatically**, which is extremely important in real-world production environments.

A template is usually a file with a `.j2` extension (for example, `nginx.conf.j2`) and contains **placeholders (variables)** written using Jinja2 syntax like `{{ variable_name }}`. When Ansible runs the playbook, it replaces these placeholders with actual values and generates the final file on the target machine.

Let's understand this with a simple example.

Here is a template file (`nginx.conf.j2`):

```
server {
    listen {{ app_port }};
    server_name {{ server_name }};

    location / {
```

```
        proxy_pass http://{{ backend_host }}:{{ backend_port }};
    }
}
```

Now, here is how you use this template in a playbook:

```
- name: Deploy Nginx config using template
  hosts: webservers
  become: yes
  vars:
    app_port: 80
    server_name: myapp.com
    backend_host: localhost
    backend_port: 8080

  tasks:
    - name: Apply nginx config
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/sites-available/myapp.conf
```

Now let's understand what happens in a real scenario. When this playbook runs, Ansible takes the `nginx.conf.j2` file, replaces all variables like `{{ app_port }}` and `{{ server_name }}` with actual values, and then creates a final configuration file on the server.

For example, the generated file will look like this:

```
server {
    listen 80;
    server_name myapp.com;

    location / {
        proxy_pass http://localhost:8080;
    }
}
```

In real-world production, templates are heavily used because different servers may require slightly different configurations. For example, one server may have a different port, another may have a different domain name, or backend services may run on different hosts. Instead of maintaining multiple static files, you use one template and control everything using variables.

Another powerful feature of templates is that you can use **conditions and loops** inside them. For example, you can dynamically generate multiple server blocks or include/exclude configurations based on environment variables.

So in simple terms, **templates allow you to create flexible and dynamic configuration files**, making your automation smarter, cleaner, and much easier to manage in real-world deployments.

Example: `index.html.j2`

```
<h1>Welcome</h1>
<p>Server IP: {{ ansible_default_ipv4.address }}</p>
```

🔹 Use Template in Task

```
- name: Copy template
  template:
    src: index.html.j2
    dest: /var/www/html/index.html
```

✔️ Same file works for all servers
✔️ Values change dynamically

🔹 Real-World Example

In companies using Ansible:

- Config files (nginx, app configs)
- Generated using templates

🔥 4. Handlers (VERY IMPORTANT)

🔹 What is a Handler?

👉 A **handler runs only when notified**

In Ansible, a **Handler** is a special type of task that runs **only when it is triggered by another task**, usually when a change occurs. This makes handlers very important in production because they help avoid unnecessary actions, like restarting services when nothing has changed.

In a normal task, Ansible executes it every time the playbook runs (unless it is already in the desired state). But a handler is different—it runs **only when notified**. This notification happens using the `notify` keyword inside a task. Handlers are commonly used for operations like **restarting services, reloading configurations, or running cleanup tasks**.

Let's understand this with a simple example:

```
- name: Handler example
  hosts: all
  become: yes

  tasks:
    - name: Update nginx config
      copy:
        src: nginx.conf
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx

  handlers:
    - name: Restart nginx
      service:
        name: nginx
        state: restarted
```

Now let's understand how this works in a real scenario. The task copies a new Nginx configuration file to the server. If the file is **changed**, Ansible triggers (notifies) the handler named "Restart nginx". The handler will then restart the Nginx service.

However, if the file is **already the same** and no change occurs, the handler will **not run at all**. This is very efficient because restarting services unnecessarily can cause downtime or performance issues in production systems.

Another important point is that handlers run **at the end of the playbook execution**, even if they are triggered multiple times. For example, if 5 tasks notify the same handler, it will still run only once.

In a real-world production scenario, imagine you are updating configuration files across multiple servers. Instead of restarting the service after every file update, handlers ensure the service is restarted **only once and only if needed**, making your deployment more stable and efficient.

So in simple terms, a **handler is a conditional task that runs only when something changes**, and it is mainly used to manage services efficiently in automation workflows.

🧠 Real Meaning

> "Restart service only if something changed"

🔹 Example

Task:

```
- name: Update config
  template:
    src: index.html.j2
    dest: /var/www/html/index.html
  notify: restart nginx
```

Handler:

```
- name: restart nginx
  service:
    name: nginx
    state: restarted
```

✔️ Efficient
✔️ No unnecessary restarts

🔥 5. Full Real-World Example (IMPORTANT)

🚀 Web Server Setup using Role

Playbook:

```
- name: Deploy Web Server
  hosts: local
  become: yes

  roles:
    - web
```

Role: `tasks/main.yml`

```
- name: Install nginx
  apt:
    name: nginx
    state: present

- name: Deploy HTML
  template:
    src: index.html.j2
    dest: /var/www/html/index.html
  notify: restart nginx
```

Role: `handlers/main.yml`

```
- name: restart nginx
  service:
    name: nginx
    state: restarted
```

👉 This is **exact real production structure**

🔹 6. Real DevOps Flow

In companies:

1. Developer pushes code
2. Jenkins triggers pipeline
3. Ansible:
    - Uses roles
    - Applies templates
    - Triggers handlers

🔹 7. Important Interview Questions

👉 Q1: What is a role?

A structured way to organize playbooks into reusable components.

👉 Q2: What is a template?

A dynamic file using variables (Jinja2).

👉 Q3: What is a handler?

A task that runs only when notified.

👉 Q4: Why roles are important?

- Reusability
- Clean structure
- Scalable for large projects

👉 Q5: When handler runs?

Only when a task reports **changed**

🔹 8. Common Mistakes (Real Debugging)

❌ Handler not running

👉 Reason:

- Task didn't change anything

❌ Template not updating

👉 Check:

- Variables correct?
- Path correct?

❌ Role not found

👉 Fix:

- Correct folder structure

🎯 Your Task (VERY IMPORTANT)

Step 1:

Create a role:

ansible-galaxy init web

Step 2:

- Install nginx
- Create template file
- Add handler

Step 3:

Run playbook

📅 Day 6 – Advanced Concepts (Loops, Tags, Includes, Vault)

🔥 1. Loops (Avoid Repetition)

🔹 Why Loops?

👉 Without loops:

- You repeat the same task multiple times ❌

👉 With loops:

- Clean + efficient code ✅

🧠 Example (Install multiple packages)

```
- name: Install packages
  hosts: local
  become: yes

  tasks:
    - name: Install multiple packages
      apt:
        name: "{{ item }}"
        state: present
      loop:
        - git
        - curl
        - nginx
```

✔️ One task → multiple operations
✔️ Cleaner playbooks

This is an **Ansible playbook used to install multiple packages using a loop**, and it's a very common real-world pattern to avoid writing repetitive tasks. Let's go step by step in a clear paragraph explanation.

The line `- name: Install packages` defines the **name of the play**, which is just a human-readable description so that when the playbook runs, you understand what this section is doing.

The line `hosts: local` tells Ansible to run this playbook on the **local machine** (instead of remote servers). This is useful when you want to configure your own system or a specific host group named `local` in your inventory.

The line `become: yes` means Ansible will use **sudo (root privileges)** to execute the tasks. This is required because installing packages needs administrative permissions.

Now, inside the `tasks:` section, we define what actions to perform. The task `- name: Install multiple packages` is just a description of the task.

The `apt:` is the module being used, which is responsible for **managing packages in Debian/Ubuntu systems**.

Now the important part is `name: "{{ item }}"`. Here, `item` is a variable that comes from the loop. Instead of writing separate tasks for each package (git, curl, nginx), Ansible uses a loop to iterate over each value and execute the same task multiple times.

The `state: present` ensures that each package is **installed if it is not already installed**. If the package already exists, Ansible will skip it, maintaining idempotency.

Then comes the `loop:` section, which contains a list:

- git

- curl

- nginx

This means Ansible will run the task three times:

1. First with `item = git`
2. Then with `item = curl`
3. Then with `item = nginx`

So internally, Ansible behaves like this:

- Install git
- Install curl
- Install nginx

In a real-world scenario, imagine you are setting up a new server for development or deployment. Instead of writing three separate tasks or manually installing each package, you use a loop to make your playbook **clean, scalable, and easy to maintain**. If tomorrow you need to add `docker`, you just add one more item to the list, and Ansible handles the rest automatically.

🔥 2. Tags (Selective Execution)

🔹 What are Tags?

👉 Tags allow you to run **only specific parts** of a playbook.

Example:

```
- name: Tagged tasks
  hosts: local

  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
      tags: install

    - name: Start nginx
      service:
        name: nginx
        state: started
      tags: start
```

This playbook demonstrates the concept of **tags in Ansible**, which are used to **control which specific tasks you want to run**, instead of executing the entire playbook every time. This is very useful in real-world scenarios where you may want to run only certain parts of your automation, like only installing packages or only starting services.

The line `- name: Tagged tasks` is just the **name of the play**, helping you understand what this playbook is about when it runs. The line `hosts: local` means the playbook will run on the **local machine**.

Now inside the `tasks:` section, the first task is `Install nginx`. It uses the `apt` module to ensure that the **nginx package is installed** using `state: present`. The important part here is `tags: install`. This means this task is labeled with the tag **install**.

The second task is `Start nginx`, which uses the `service` module to ensure that the **nginx service is running** using `state: started`. This task is labeled with the tag **start**.

Now the key concept is how tags are used during execution. By default, if you run the playbook normally, **both tasks will execute**. But if you want to run only specific tasks, you can use tags from the command line.

For example, if you run:

```
ansible-playbook playbook.yml --tags install
```

only the **Install nginx** task will run, and the start task will be skipped.

Similarly, if you run:

```
ansible-playbook playbook.yml --tags start
```

only the **Start nginx** task will execute.

In a real-world production scenario, imagine you have a large playbook with 50 tasks. Sometimes you only want to install software, sometimes only restart services, and sometimes only update configurations. Instead of running everything, tags allow you to **execute only the required part**, saving time and avoiding unnecessary changes.

So in simple terms, **tags act like filters for tasks**, giving you flexibility and control over what part of the playbook should run.

Run only install:

```
ansible-playbook playbook.yml --tags install
```

✔️ Useful in production
✔️ Saves time

🔥 3. Includes & Imports (Modular Code)

🔹 Why?

👉 Break large playbooks into smaller files

In Ansible, **Includes and Imports are used to split large playbooks into smaller, reusable files**, making your code more modular, clean, and maintainable. In real-world production projects, instead of writing one huge playbook with hundreds of tasks, you break it into multiple files and then combine them using includes or imports.

The main idea is simple: instead of writing everything in one place, you **reuse and organize tasks, playbooks, or roles into separate files** and load them when needed.

Now, there are two important concepts here: **import** and **include**, and the key difference between them is **when they are processed**.

Imports are static, which means they are processed at the time Ansible reads the playbook (before execution starts). This makes them predictable and slightly faster. For example:

```
- name: Main playbook
  hosts: all

  tasks:
    - name: Import tasks
      import_tasks: install.yml
```

Here, `install.yml` is loaded completely before execution begins. Ansible treats it as if all tasks inside `install.yml` were written directly in this playbook.

On the other hand, **Includes are dynamic**, which means they are processed **during execution time**. This allows you to use conditions, loops, and variables more flexibly.

```
- name: Main playbook
  hosts: all

  tasks:
    - name: Include tasks dynamically
      include_tasks: install.yml
```

Here, `install.yml` is loaded only when this task is reached during execution.

Now let's understand this with a real-world example. Imagine you are deploying an application, and your setup has different parts like installing packages, configuring the app, and starting services. Instead of writing everything in one file, you create separate files like:

- `install.yml`
- `configure.yml`
- `deploy.yml`

Then your main playbook looks like this:

```
- name: Production deployment
  hosts: webservers

  tasks:
    - import_tasks: install.yml
    - import_tasks: configure.yml
    - import_tasks: deploy.yml
```

This makes your code **clean and easy to manage**, especially when multiple developers are working on the same project.

In production, **imports are generally preferred for predictable workflows**, while **includes are used when you need flexibility**, such as running tasks conditionally or looping over task files.

So in simple terms, **imports and includes help you break large automation into smaller reusable pieces**, improving readability, maintainability, and scalability of your Ansible code.

Example:

`main.yml`

```
- name: Main playbook
  hosts: local

  tasks:
    - import_tasks: install.yml
    - import_tasks: config.yml
```

`install.yml`

```
- name: Install nginx
  apt:
    name: nginx
    state: present
```

✔️ Cleaner structure
✔️ Easier debugging

🔥 4. Ansible Vault (SECURITY – VERY IMPORTANT)

Ansible Vault is a security feature in Ansible used to **encrypt sensitive data** such as passwords, API keys, SSH keys, and database credentials inside your playbooks or variable files. In real-world production environments, you should never store secrets in plain text, and Ansible Vault solves this problem by keeping them **secure and encrypted** while still allowing Ansible to use them during execution.

When you use Ansible Vault, the sensitive data is stored in an **encrypted format**, and only someone with the correct password (vault password) can view or use it. This ensures that even if your code is stored in GitHub or shared with a team, your secrets remain protected.

Let's understand this with a simple example.

Suppose you have a variable file `secrets.yml`:

```
db_password: mysecret123
```

This is unsafe because the password is visible. To secure it, you encrypt the file using:

```
ansible-vault encrypt secrets.yml
```

After encryption, the file will look something like this:

```
$ANSIBLE_VAULT;1.1;AES256
623734623834...
(encrypted content)
```

Now the password is no longer readable. To use this encrypted file in a playbook:

```
- name: Use vault example
  hosts: all
  vars_files:
    - secrets.yml

  tasks:
    - name: Print password
      debug:
        msg: "{{ db_password }}"
```

When you run this playbook, Ansible will ask for the **vault password**, decrypt the file temporarily, and use the value securely.

In real-world production scenarios, this is extremely important. For example, when deploying applications, you may need database passwords, cloud credentials, or API tokens. Instead of exposing them in code, you store them in vault-encrypted files and safely use them in your automation.

There are also other useful commands like:

- `ansible-vault view secrets.yml` → to view encrypted content
- `ansible-vault edit secrets.yml` → to edit securely
- `ansible-vault decrypt secrets.yml` → to remove encryption

So in simple terms, **Ansible Vault is used to protect sensitive information by encrypting it**, ensuring your automation is secure, production-ready, and safe to share or store in version control systems.

🔥 5. Real-World Example (Combining Everything)

🚀 Advanced Playbook

```yaml
- name: Advanced Playbook
  hosts: local
  become: yes

  vars_files:
    - secrets.yml

  tasks:
    - name: Install packages
      apt:
        name: "{{ item }}"
        state: present
      loop:
        - nginx
        - git
      tags: install

    - name: Copy config
      template:
        src: index.html.j2
        dest: /var/www/html/index.html
      notify: restart nginx
      tags: config
```

```
handlers:
  - name: restart nginx
    service:
      name: nginx
      state: restarted
```

🔹 6. Real DevOps Scenario (VERY IMPORTANT)

In companies using:

- Jenkins
- Amazon Web Services

👉 Workflow:

1. Pipeline triggers

Run only specific tags:
--tags deploy

2.
3. Secrets loaded from Vault
4. Multiple servers updated using loops

🔹 7. Important Interview Questions

👉 Q1: What is loop in Ansible?

Used to execute a task multiple times with different values.

👉 Q2: What are tags?

Used to run specific parts of playbook.

👉 Q3: Difference between import and include?

import	include
Static	Dynamic

Loaded at start Loaded at runtime

👉 Q4: What is Ansible Vault?

Used to encrypt sensitive data.

👉 Q5: Why use Vault?

To protect passwords, API keys, and secrets.

🔹 8. Common Mistakes (Real Debugging)

❌ Loop not working

👉 Check:

- `{{ item }}` used correctly?

❌ Tag not running

👉 Check:

- Correct tag name?

❌ Vault error

👉 Fix:

```
--ask-vault-pass
```

🎯 Your Task (IMPORTANT)

Step 1:

- Create playbook using loop
- Add tags

Step 2:

- Encrypt a file using vault

Step 3:

Run:

ansible-playbook playbook.yml --tags install

📅 Day 7 – Real-World Ansible + DevOps Integration

🔥 1. How Companies Actually Use Ansible

In real companies, Ansible is NOT used alone.

It works with:

- Jenkins (CI/CD)
- Amazon Web Services (servers)
- Docker / Kubernetes

🔁 Real DevOps Flow (Very Important)

🧠 Step-by-step:

1. Developer pushes code → GitHub
2. Jenkins pipeline starts
3. Build happens (Java / Spring app)
4. Ansible runs playbook:
    - Install dependencies
    - Copy application
    - Restart service
5. App goes live 🚀

🔥 2. Real-World Project (Spring Boot Deployment)

Since you're a Java developer, this is exactly what you'll do in companies.

🧾 Scenario:

👉 You want to deploy a **Spring Boot JAR**

🚀 Playbook Example

```yaml
- name: Deploy Spring Boot App
  hosts: local
  become: yes

  vars:
    app_name: myapp.jar
    deploy_path: /opt/myapp/

  tasks:
    - name: Install Java
      apt:
        name: openjdk-11-jdk
        state: present

    - name: Create directory
      file:
        path: "{{ deploy_path }}"
        state: directory

    - name: Copy JAR
      copy:
        src: "{{ app_name }}"
        dest: "{{ deploy_path }}{{ app_name }}"

    - name: Start application
      shell: "nohup java -jar {{ deploy_path }}{{ app_name }} &"
```

✔️ This is a **real production-level deployment**

🔥 3. Rolling Updates (Zero Downtime)

🔹 What is Rolling Update?

👉 Update servers one by one instead of all at once

Example:

```
- name: Rolling update
  hosts: web
  serial: 1

  tasks:
    - name: Restart app
      service:
        name: myapp
        state: restarted
```

✔️ No downtime
✔️ Used in production

🔥 4. Error Handling (VERY IMPORTANT)

Example:

```
- name: Install nginx
  apt:
    name: nginx
    state: present
  ignore_errors: yes
```

Retry Example:

```
- name: Retry service start
  service:
    name: nginx
    state: started
  retries: 3
  delay: 5
```

🔥 5. Real Production Issues + Debugging

❌ Issue 1: Playbook fails on one server

👉 Fix:

- Use `ignore_errors`
- Check logs:

```
ansible-playbook playbook.yml -vvv
```

❌ Issue 2: Application not starting

👉 Debug:

```
ps -ef | grep java
netstat -tulnp
```

❌ Issue 3: SSH failure

👉 Fix:

- Check key
- Check user
- Test manually:

ssh user@server

❌ Issue 4: Service not restarting

👉 Check:

- Handler triggered?
- Config changed?

🔥 6. Scaling to 1000+ Servers

In companies:

- Inventory is dynamic (cloud-based)
- Ansible runs parallel tasks

Example:

- 500 servers updated in minutes

🔥 7. Ansible vs Other Tools (Interview)

👉 Ansible vs Terraform

Ansible	Terraform
Config management	Infrastructure provisioning
Mutable	Immutable
Uses SSH	Uses APIs

👉 Example:

- Terraform → create EC2
- Ansible → configure EC2

🔥 8. MOST IMPORTANT INTERVIEW QUESTIONS

👉 Q1: How Ansible works in CI/CD?

It is triggered by tools like Jenkins to deploy applications automatically.

👉 Q2: How do you handle failures?

- Logs (`-vvv`)
- Retry
- ignore_errors

👉 Q3: What is rolling update?

Updating servers one by one to avoid downtime.

👉 Q4: How do you secure secrets?

Using Ansible Vault.

👉 Q5: How to scale Ansible?

- Dynamic inventory

- Parallel execution

🔥 9. Real Architecture (Visualization)

👉 In a company:

- GitHub (code)
- Jenkins (pipeline)
- Ansible (deployment)
- Amazon Web Services (servers)

🎯 Final Hands-On Project (DO THIS)

🚀 Build this:

1. Install Java
2. Create directory
3. Copy dummy JAR
4. Start app

👉 Bonus:

- Add handler
- Add variable
- Add tag

🧠 What You Achieved in 7 Days

You now know:

- Playbooks
- Roles
- Variables
- Vault
- Real deployment

👉 This is **industry-level knowledge**

🔥 REAL PRODUCTION INCIDENTS (Ansible + DevOps)

🚨 Incident 1: Deployment Failed on Few Servers

🧠 Scenario

You run Ansible playbook:

```
ansible-playbook deploy.yml
```

👉 Result:

- 8 servers → ✅ success
- 2 servers → ❌ failed

🔍 Step-by-Step Debugging (REAL FLOW)

1. Increase verbosity

```
ansible-playbook deploy.yml -vvv
```

👉 You see:

FAILED! => SSH timeout

2. Root Cause

👉 Possible reasons:

- Server down
- SSH blocked
- Wrong key

3. Verify manually

```
ssh user@server-ip
```

4. Fix

- Restart server OR
- Fix SSH key OR

- Update inventory

💡 Real DevOps Thinking

👉 Never assume playbook is wrong first
👉 Always check **infrastructure issue**

🚨 Incident 2: Application Not Starting After Deployment

🧠 Scenario

Ansible deployed your Spring Boot JAR, but:

curl localhost:8080

👉 ❌ Not working

🔍 Debug Flow

1. Check process

```
ps -ef | grep java
```

👉 Output:

(no process found)

2. Check logs

```
cat nohup.out
```

👉 Output:

Port already in use

3. Root Cause

👉 Another app is already using port 8080

4. Fix

```
lsof -i :8080
kill -9 <PID>
```

OR change port

💡 DevOps Insight

👉 Deployment success ≠ Application success
👉 Always verify **app health**

🚨 Incident 3: Playbook Runs But Changes Not Applied

🧠 Scenario

You updated config file, but:

- App still using old config ❌

🔍 Debug Flow

1. Check output

ok=5 changed=0
👉 Means:

- Ansible thinks nothing changed

2. Root Cause

👉 Idempotency issue:

- File content same OR
- Template not updated

3. Fix

- Verify template changes
- Force update:

force: yes

💡 Insight

👉 Ansible is **state-based**, not command-based
👉 It only changes when needed

🚨 Incident 4: Handler Not Triggered

🧠 Scenario

Config updated but service not restarted

🔍 Debug

1. Check task output

👉 If:

changed=0

👉 Handler will NOT run

2. Root Cause

👉 No actual change detected

3. Fix

- Ensure file changed
- Use correct template

💡 Insight

👉 Handlers run only on **change events**

🚨 Incident 5: Permission Denied Error

🧠 Scenario

FAILED! => Permission denied

🔍 Debug

1. Check user

```
whoami
```

2. Fix

--become

or in playbook:

become: yes

💡 Insight

👉 Most production issues = **permission problems**

🚨 Incident 6: Package Installation Fails

🧠 Scenario

FAILED! => Unable to locate package nginx

🔍 Debug

1. Check OS

```
ansible all -m setup | grep os_family
```

2. Root Cause

👉 Wrong module:

- Debian → `apt`
- RedHat → `yum`

3. Fix

when: ansible_os_family == "Debian"

💡 Insight

👉 Always write **OS-aware playbooks**

🚨 Incident 7: Inventory Issue (Wrong Server)

🧠 Scenario

Deployment happened on wrong server 😱

🔍 Debug

1. Check inventory

cat inventory.ini

2. Verify target

ansible web -m ping

3. Fix

- Correct group

- Use proper naming

💡 Insight

👉 Inventory mistakes = **very common in real world**

🚨 Incident 8: Secrets Exposed (Critical)

🧠 Scenario

Password visible in playbook 😨

🔍 Fix

Use:

ansible-vault encrypt secrets.yml

💡 Insight

👉 Never store secrets in plain text
👉 Always use **Vault**

🔥 REAL DEVOPS MINDSET (VERY IMPORTANT)

🧠 Always Follow This Debug Flow:

1. Check error output (`-vvv`)

2. Verify manually (SSH, logs)

3. Identify:

- Infra issue?
- App issue?
- Ansible issue?

🧠 Golden Rule:

👉 70% issues = Infrastructure
👉 20% issues = Configuration
👉 10% issues = Ansible

🔥 REAL COMPANY FLOW

In companies using:

- Jenkins
- Ansible

🚀 Incident Handling:

1. Alert triggered 🚨
2. Check logs
3. Run Ansible debug
4. Fix issue
5. Redeploy

🎯 YOUR PRACTICE TASK (VERY IMPORTANT)

Simulate these:

✅ Scenario 1:

- Start app on port 8080
- Start another app → see failure

✅ Scenario 2:

- Remove `become` → see permission error

✅ Scenario 3:

- Break inventory → debug